

iOS app-data versioning

goStoriesAI — documentation interne

Axel Fortunato — LICENCESINFO

How local data stays compatible across app updates: schema changes, default-story reseeds, and data backfills — all behind **one** version number.

What it covers

A single integer `app_data_version` tracks how far this install has been upgraded. Each migration step can do any combination of:

- **Schema** — new columns or tables in the local database
- **Bundled defaults** — replace the stories shipped with the app
- **Data backfill** — fill new fields for existing rows, locally or from the API

Stories imported from the catalog are only touched when a migration explicitly says so. This does **not** replace App Store binary versioning: it applies to local data only.

How it works on launch

1. Read `app_data_version` on the device; if absent, treat it as 0.
2. While the local version is lower than the one expected by the installed build, run the migration whose `toVersion` equals `local + 1`.
3. Write the new version number **only after the step succeeds**.
4. If a step throws, the version stays at $N - 1$, so the next launch retries the same step instead of skipping it.

Example: a device installed at v1 receiving a build that ships `currentVersion = 3` runs $1 \rightarrow 2$, then $2 \rightarrow 3$.

Safety rules

- Never delete a migration that has already shipped.
- Bump `app_data_version` only after success.
- Keep every step idempotent, so retries are safe: check before altering, and fill only rows where the value is still missing.
- Keep network work retry-friendly — on failure, do not bump.
- One global version number: no separate counters for schema and defaults.

Release checklist

- ☐ Migration written and idempotent
- ☐ Registered in the migration list, `currentVersion` bumped
- ☐ Public version aligned with the Xcode marketing version
- ☐ Fresh-install schema updated to match the latest shape
- ☐ Fresh install, upgrade, and interrupted-migration paths tested